08 HANDS-ON WORKSHOP

Build a Speech-to-Speech Study Agent



A voice assistant you can extend in Python
You only need basic Python and ML knowledge

Aisyah Izzah | Data Scientist, Fata Organa

35 minutes | Work in groups of 3

What we will build

Say a request

“Tambahkan tugas belajar agentic AI,
lalu tampilkan daftarnya.”

See and hear the result

The agent adds a task and checks the list.
You hear its reply through TTS.

Your goal: add a capability to a voice agent and explain how it works.

A cascaded speech-to-speech system

One recorded request, one spoken reply. Each stage remains visible.

Speech input
Record or upload

ASR
Audio becomes text

Agent + tools
Choose, run, check

TTS
Reply becomes audio

Speech output
Press Play to listen

The agent receives the actual tool results

The model writes a reply from tool feedback. TTS reads that reply aloud.

Separate ASR, agent, and TTS components. Recorded turns, not continuous live audio.

Our 35-minute workshop

3 min	Form groups, open the notebook, and run setup
5 min	Record a request and inspect the transcript
10 min	Follow the tool trace and hear the reply
12 min	Implement and test count_tasks()
5 min	Discuss one result and prepare your submission

What you need

Notebook	Download the notebook here
Colab	Upload it to Colab; we install packages together
GPU runtime	Runtime → Change runtime type → T4 GPU
Audio	Microphone, speakers, and one short practice run
Fallback	INPUT_MODE = "text" or SPEAK = False

Step 1: setup and load the models

Notebook steps 1–2

Choose a T4 GPU runtime.
Run the install cell.
Run the model-loading cell.

Checkpoint

“Models ready on cuda” appears.
First load downloads ~2 GB.
No API key: models run in Colab.

Your first audio input also downloads the ASR model.

Step 2: speech becomes text

Notebook step 3

```
INPUT_MODE = "record"
LANGUAGE = "id"

# Fallback: "upload" or "text"
```

Checkpoint

Read the transcript aloud.
Does it match the recording?
Correct errors before continuing.

Run the input cell, click Record, allow the mic, speak, then click Stop.

Step 3: Python tools change the state

Available tools

```
add_task(task)
show_tasks()
study_tips(topic)
```

Try without the model

```
add_task("Belajar Python")
print(show_tasks())

# Reset before the full demo:
tasks.clear()
```

Keep state visible: tasks stores the list; study_tips is a fixed template.

Step 4: the model needs a tool description

Notebook step 5

Name: add_task
Purpose: add a study task
Argument: task (string)
Required: task

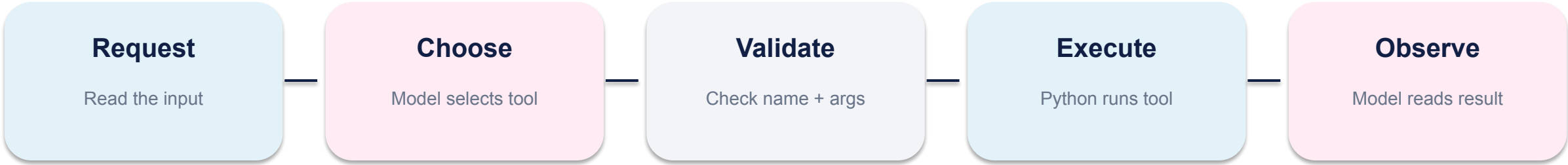
Example tool request

```
name = "add_task"
arguments = {
    "task": "Belajar Python"
}
```

A tool definition describes a function. Python still executes it.

Step 5: the agent uses tool feedback

Notebook step 6: inspect ask_agent() and execute_tool()



A bounded agent loop

Two tool rounds support add + show, followed by a final text response.

Each new request starts fresh chat context. The task list persists.

Checkpoint: two tools, one request

Example trace

1. add_task("Belajar agentic AI")
2. show_tasks()

The trace should show both calls.

Check the real state

print(show_tasks())

Expected after a clean start:
1. Belajar agentic AI

Notebook step 8: run one turn once. Replay audio without repeating the action.

Step 6: the reply becomes speech

Notebook step 7

speak_reply(last_reply)

Input: final reply text
Output: audio player

Checkpoint

Press Play to hear the reply.
Compare the voice with the text.
The voice is AI-generated.
TTS does not verify the action.

If synthesis fails, retry TTS only. The agent action may already be complete.

Main exercise: add count_tasks()

TODO 1: Python function

```
def count_tasks():
    # Return the count as a string.
    # Keep the task list unchanged.
    ...
```

TODOs 2–3: connect it

Describe the tool in its schema.
Use the exact name count_tasks.
Accept no arguments.
Add its branch in execute_tool().

You have 12 minutes, including the tests on the next slide.

Test your new tool

Local checks, no model

Empty list: count is "0"
One task: count is "1"
Two tasks: count is "2"

Counting must not change the list.

Test with the agent

“Ada berapa tugas saya?”

Confirm count_tasks in the trace.
Compare the reply with len(tasks).
Listen to the spoken answer.

Finished early? Try complete_task(task_id) and include your attempt in the submission.

Submission challenge: complete a task

Try	“Tandai tugas nomor satu sebagai selesai.”
State	Store task + completed; display pending or done.
Tool	Add complete_task(task_id); keep total count correct.
Checks	Test invalid task numbers and already-completed tasks.
Submit	Include your attempt. Date challenge: self-study only.

When something goes wrong

Setup	Run prior cells; use a T4 GPU runtime.
Input audio	Use Record/Stop, upload, or INPUT_MODE = "text".
Wrong count	Check the function, schema name, and dispatcher.
No sound	Press Play; retry speak_reply(last_reply) only.
Duplicates	Inspect show_tasks() before repeating an agent request.

What did you build today?

Explain the components

ASR turns speech into text.
The agent chooses and checks tools.
Python validates and executes them.
TTS turns the reply into speech.

Before a real product

Handle unclear requests.
Confirm consequential actions.
Prevent duplicate actions.
Add persistent storage if needed.

Can a fluent spoken reply still describe the wrong action?

Before you submit

Notebook	Share one notebook link per group.
Access	Make sure the instructor can open the link.
Evidence	Keep the check output and one tool trace visible.
Reflection	Answer three questions in Bahasa Indonesia or English.
Form	Submit everything through the Google Form.